Relazione progetto IOT #03 - Smart Dam

Fabio Solaroli

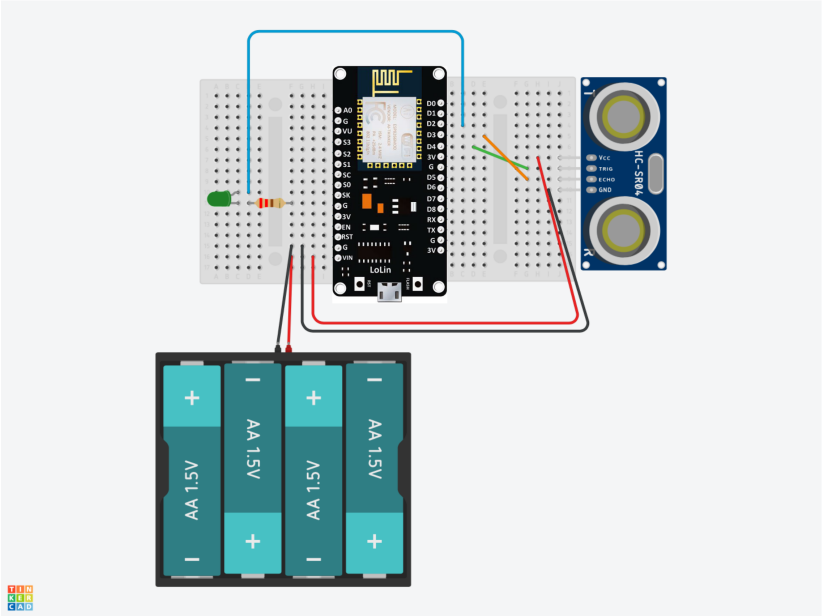
La consegna richiedeva la creazione di un sistema che simulasse la gestione di una diga.
Il sistema è suddiviso in cinque sottosistemi differenti.

Remote Hydrometer

Ha il compito di misurare il livello dell'acqua della diga.

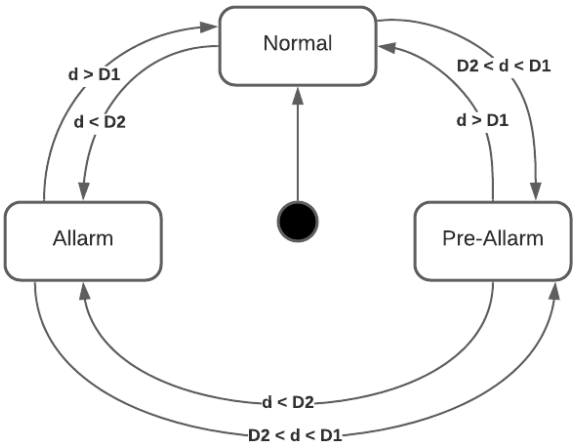
Hardware

Il sottosistema è stato realizzato usando un ESP8266 a cui è collegato un led, con il compito di indicare lo stato del sistema, e un sensore di distanza a ultrasuoni, per la misurazione del livello dell'acqua.
Essendo che il sensore richiedeva un'alimentazione da 5V, ho alimentato tutto il sottosistema attraverso un'alimentazione esterna, siccome ESP fornisce al massimo 3,3V.



Software

Il sottosistema si basa su una macchina a strati finiti, cha cambia in base al livello dell'acqua misurato.



Il comportamento è stato suddiviso in diversi task, guidati da uno scheduler:

- BlinkTask: si occupa di gestire la visualizzazione dello stato del sistema attraverso il led;
- SonarMeasureTask: si occupa di misurare il livello dell'acqua attraverso il sensore a ultrasuoni e cambia lo stato del sistema in base ad esso;
- MQTTCommunicationTask: si occupa di comunicare con il Dam Service, attraverso il protocollo MQTT, comunicando lo stato del sistema e il livello dell'acqua misurato, nel caso si sia in stato di pre-allarme o allarme. Durante lo stato normale il sottosistema comunica comunque il suo stato, così da assicurarsi che la comunicazione continui ad essere attiva e non avvengano guasti o errori nel mezzo di comunicazione.

Per gestire l'interruzioni dei timer per lo scheduler su ESP ho utilizzato la libreria "ESP8266TimerInterrupt".

Dam Service

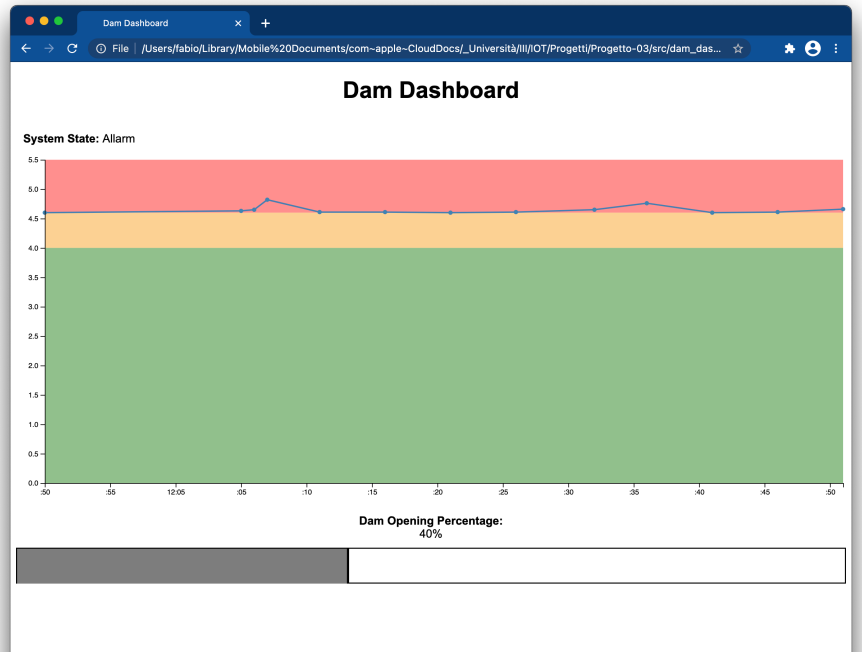
Ha il compito di unire i vari sottosistemi tra loro, facendo da tramite per lo scambio e l'inoltre di informazioni.

Raccoglie i dati sullo stato del sistema e il livello dell'acqua che arrivano dal Remote Hydrometer attraverso il protocollo MQTT, e attraverso questi dati calcola la percentuale di apertura della diga. Comunica inoltre al Dam Controller come comportarsi attraverso la seriale. Nel caso lo stato del sistema sia stato "sovrascritto" dallo stato manuale, allora dovrà anche leggere dal Dam Controller il livello di apertura della diga che sarà a sua volta stato indicato dalla Dam Mobile App. Infine risponde alle richieste HTTP della Dam Dashboard comunicando lo stato di tutto il sistema.

Il server MQTT e la gestione delle richieste HTTP sono basate sulla libreria Verx, mentre la comunicazione seriale sulla libreria Jssc. Tutto il sottosistema è stato realizzato in Java.

Dam Dashboard

La dashboard è stata implementata come web app attraverso HTML e Javascript; i dati inviati ad essa sono in formato JSON, e vengono utilizzati per creare il grafico dell'andamento del livello dell'acqua attraverso la libreria d3.js.

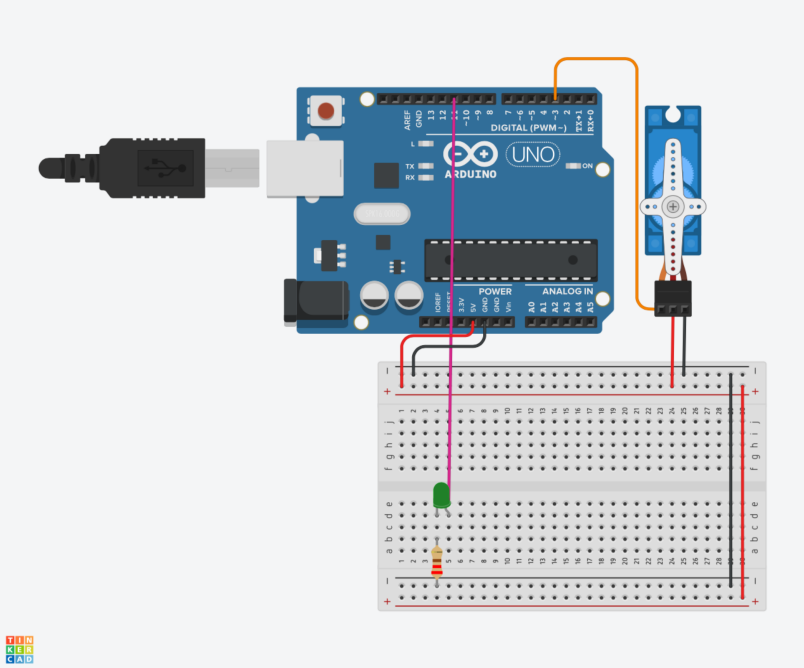


Dam Controller

Ha il compito di controllare l'apertura della diga.

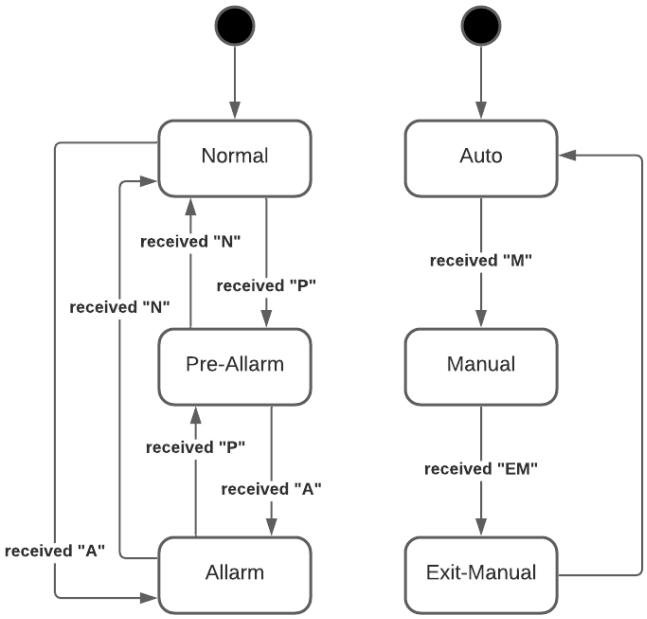
Hardware

Il sottosistema è realizzato con Arduino a cui è collegato un led che ha il compito di visualizzare lo stato del sistema e un servo motore, che simula la diga.
Purtroppo non possedendo un dispositivo Android non ho potuto utilizzare la comunicazione con la Dam Mobile App attraverso il modulo di comunicazione bluetooth, ma sono dovuto ricorrere all'emulatore di Android Studio, che non essendo in grado di utilizzare la comunicazione bluetooth mi ha costretto ad utilizzare il bridge emulatore-seriale fornito.



Software

Il sottosistema si basa su una macchina a strati finiti, che cambiano in base ai dati inviati dal Dam Service o alla possibile sovrascrittura in manuale attivata dalla Dam Mobile App.



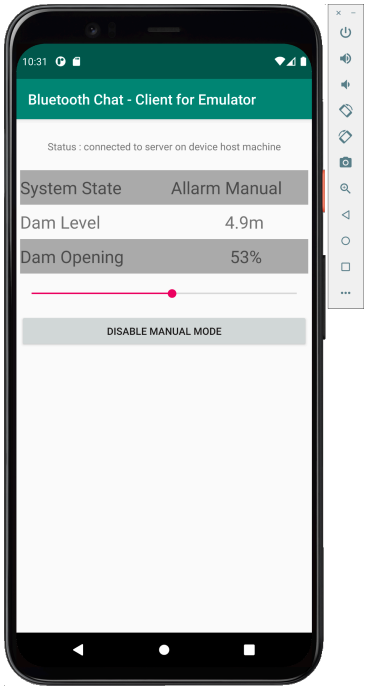
Il comportamento è suddiviso in task, gestiti da uno scheduler:
- BlinkTask: si occupa di gestire la visualizzazione dello stato del sistema attraverso il led;
- DamOpeningTask: si occupa di gestire il movimento del servomotore;
- SerialReceiveTask: si occupa di gestire la ricezione dei messaggi che arrivano dal Dam Service attraverso la comunicazione seriale, così da aggiornare il proprio stato interno e impostare il livello di apertura della diga corretto;
- SerialSendTask: si occupa di inviare i messaggi sul canale seriale, indicando al Dam Service i livelli di apertura della diga aggiornati nel caso di modalità manuale;
- BTReceiveTask: si occupa di gestire la ricezione dei messaggi che arrivano dal canale bluetooth simulato, per ricevere istruzioni sull'entrare in modalità manuale e, nel caso, l'apertura indicata dalla Dam Mobile App;
- BTSendTask: si occupa di gestire l'invio dei messaggi sul canale bluetooth simulato, inoltrando il messaggio ricevuto da seriale.

Inizialmente i 4 task per gestire la comunicazione erano raggruppati in due soli task, uno per la comunicazione con il canale bluetooth simulato e l'altro per la comunicazione seriale con il Dam Service, ma questo creava inconsistenze nell'ordine delle operazioni e, sopratutto, rendeva molto lenta la comunicazione tra la Dam Mobile App e il Dam Controller.
Scorporando i due macro-task i task più piccoli ho potuto aumentare la "velocità di comunicazione" (diminuendo il periodo dei task) dalla Dam Mobile App, passando per il Dam Controller, verso il Dam Service, lasciando la comunicazione in senso contrario più "lenta"; i dati arrivano infatti dal Remote Hydrometer al massimo una volta ogni cinque secondi, ed non è quindi necessario inoltrare i dati dal Dam Service verso la Dam Mobile App ad una frequenza maggiore.

Dam Mobile App

Ha il compito di visualizzare i dati dello stato dell'intero sistema, dando anche la possibilità, nel caso di sistema in modalità di allarme, di entrare in modalità manuale e specificare il livello di apertura della diga.

In particolare la modalità manuale può essere, come detto, attivata soltanto in caso di stato d'allarme del sistema generale e viene disattivata soltanto nel caso di input specifico dell'utente o nel caso in cui si chiuda l'applicazione.



Come detto il precedenza non essendo in possesso di un dispositivo Android il sottosistema è stato realizzato sull'emulatore di Android Studio e utilizzando la libreria fornita per emulare il canale bluetooth. Ho eseguito qualche piccola modifica su questa libreria per gestire in maniera migliore la disconnessione e riconnessione del dispositivo emulato dal Dam Service senza che il sistema entrasse in errore.
Ho riportato tutte le modifiche fatte nel file "btlib-PatchFile", inserito all'interno della cartella "doc".

Non ho inoltre implementato la comunicazione diretta attraverso HTTP con il Dam Service e quindi, come detto prima, tutti i dati passano attraverso il Dam Controller.